

A Novel Compression Technique For DNA Sequence Compaction

Ratnakumari Challa
Dept of CSE
RGUKT, RK Valey,
Kadapa, India.

Pranayani Devi.G
Dept of CSE
JNTUK,
Kakinada, India

Karuna Arava
Dept of CSE
JNTUK,
Kakinada, India.

Kanusu SrinivasaRao
Dept of CSE
YV University,
Kadapa, India

Abstract—Modern Biotechnology produces large amount of genomic data. The explosion of DNA data has given a challenge for understanding genomic structure, the disk storage and computation. It is essential for the development of efficient compression techniques to handle genomic data storage. Data compression is used to store the data in less memory. The properties of DNA sequence offer a chance to build DNA specific compression algorithms. In this paper, a novel compression technique is proposed for genomic data. In the first stage, each base in DNA sequence is converted into binary form using 2-bit encoding system. On the resultant binary string, A Modified run length encoding is applied. The output is compressed again using Huffman encoding technique in second stage. The encoded sequence is converted into ASCII characters. This technique is quite simple and effective.

Keywords—Compression, DNA sequence, Huffman coding, Run length encoding

I. INTRODUCTION

A. Compression

As the technology grows, Everyday quintillion bytes of data are being created globally. The problem of storing large data gives two problems in terms of time and space. The space required to store and the time for encoding and decoding the data. As the data increases, the complexity for processing the data also increases. Hence to store and process the data in an effective way compression is needed [1]. Compression techniques are related with the type of the data. There are various compression techniques to compress the different types of data such as text, image, and video. WinZip, JPEG, MPEG are some well-known compressors. Compression techniques can be classified into two types either lossless or lossy compression. Lossy compression [2] involves some loss of information and the original data can't be recovered exactly. Lossy compression is widely used for image and video and audio file compression. JPEG, MPEG, GIF are some lossy compression techniques. Lossless compression technique reduces the bits by eliminating the redundant information. Lossless compression involves no loss of information and the original data can be recovered from the compressed data. Lossless compression mainly used are gzip, Huffman coding, Run length encoding, Arithmetic coding, Shannon-fanno coding.

B. DNA sequence Compression

In a cell, DNA (Deoxyribonucleic Acid) is a molecule that carries the genetic information. DNA is in double helix structure in which two individual DNA strands twist around each other in a spiral. The DNA strand consists of four nucleotide bases Adenine, Cytosine, Guanine and Thymine abbreviated as A, C, G, T [3]. DNA sequence is the combination of these four bases only. The human genome is made up of 3 billion of these genetic characters. DNA sequences are useful in areas such as genetics, biology, and medicine. In research, genomic data leads to better understanding of cell functions and organism evaluation. In modern life science research, collecting and analyzing genome for all organisms and their species has become cornerstone. DNA sequences can be tremendous in size. The genomes of many species are now available in public databases like Genbank. The database's sizes are increasing exponentially day by day. Hence to store, access and process this genome data is a challenging task. Transferring and storage of data is quite expensive. We design and implemented a new compression technique to achieve high compression ratio [4].

II. RELATED EXISTING ALGORITHMS

A. Two-Bit encoding

Universal compression algorithms are not suitable for genome sequence. Genome sequence consists of only four nucleotides. Two bits are enough to encode the four characters. Binary bits are assigned for every base of DNA as mentioned below, which gives a binary sequence as an output.

$$A=00, C=01, G=10 \text{ and } T=11$$

B. Run length Encoding

Run length encoding is a simple lossless compression technique in which runs of data are stored in a single value. The run can be defined as count of consecutive occurrence of a data symbol. RLE is most useful on data that contains many runs and repetition. For Example, The following sequence:

AAAAAAAAATTTTCCGGG

and its encoded result is A9T5C2G[5].

C. Modified Run length Encoding

The binary sequence is consists of alternating runs of Zeros and alternating runs of ones. In encoding, an additional parameter is associated with the sequence of runs. If $a=0$, the run length is treated as runs of 0's. Otherwise, if $a=1$, the run length is considered as runs of 1's. The binary data can be classified into runs of zero's ending with a one and runs of one's ending with a zero and runs of one's ending with zero. For example:

Original data: 0 0 1 1 1 1 1 0 0 0 0 1 1 1 1 0 1 1 1 1 1 0
 Run lengths: 2 4 3 3 5
 Parameter a : 0 1 0 1 1

D. Huffman coding

Huffman code is the variable length code to map fixed length code. A table is derived from the estimation of probability or frequency of occurrence for each source symbol. Based on the list of all symbols which are arranged in descending order of probabilities Huffman tree is constructed. The principle used for encoding is to use lower number bits for the data that occurs more frequently [6].

E. ASCII encoding

ASCII encoding technique is used to convert the binary data into text format. 8-bit binary is converted into decimal number which can be turned as equivalent ASCII code.

III. PROPOSED MODEL

Double compression technique is proposed in this model. First, the binary data is given as input to the first level where data is compressed by using modified run length encoding. On the compressed data, another level of compression is taken place by using Huffman code. Huffman code is constructed by the frequency of more number of repeated data values from the first level compressed data. Based on the frequency, Huffman tree is constructed and then a variable length code word is formed. The flow of proposed model is given in the figure Fig.1.

A. Algorithm:

Step 1: convert each base into binary code using 2-bit encoding.

Step 2:

(i). Apply modified run length encoding technique on that binary sequence.

(ii). A sequence of run lengths will be the output.

Step 3: Calculate the frequencies of each run length throughout the sequence.

Step 4: Construct Huffman tree based the frequencies.

Step 5: Assign variable length code for each run length character using Huffman coding.

Step 6: Replace those variable length codes in the run length sequence.

Step 7: Convert the binary sequence which is the output of step 6 into ASCII characters.

Step 8: Calculate the compression ratio as the ratio

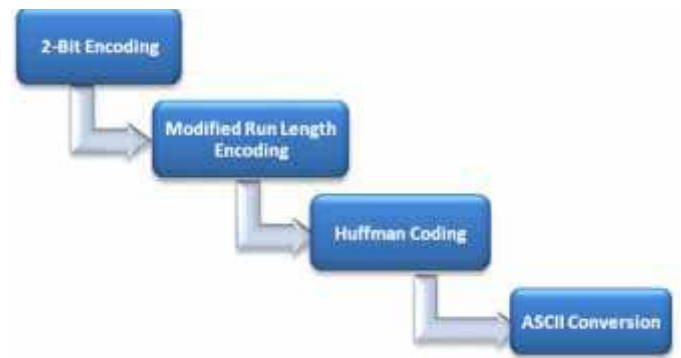


Fig1: Proposed model for DNA sequence compaction

Compression Ratio = (No. of ASCII Characters*8)/ No. of bases in sequence.

B. Explanation of proposed model with an example

In this, a double compression technique on DNA sequence is applied. The DNA sequence compression is done in two phases.

Step 1: In first phase the sequence is converted to binary string using 2 bit encoded scheme. A sub string of a DNA sequence is considered:

AGATCAAGTTTTATTTC AATATTTCTACCTCATTTCTGAT
 TCTTATTT

Assign 2 bits for every DNA base, then the encoded string become binary sequence as follows:

001000110100001011111111100111111010000110011111
 01110001011101001111101111000111101111001111

Step 2: Now apply Modified run length encoding on the resultant binary sequence to find out the runs of 0's and 1's. The sequence of runs will be given as follows:

2 3 1 1 3 1 9 1 5 1 3 1 1 5 3 2 1 2 1 1 5 4 2 3 1 4 1 3

Step 3: The sequence of runs is then compressed by using Huffman encoding technique [7]. Prefix free codes are allotted to the runs based on their frequencies as given in the Table 1.

Step 4: Assign the variable length codes to run lengths

110 10 00 10 0 11110 0 1110 0 10 0 0 1110 10 110 0 110 0 0
 1110 11111 110 10 0 11111 0 10

Step 5: The resultant binary sequence is undergone on ASCII encoding technique

11010001 00111100 11100100 01110101 100110001
 11011111 11010011 111010 000 (appended bits)

Table1 : Prefix free code allocation

Symbol	Frequency	Variable length code
1	12	0
3	6	10
2	4	110
5	3	1110
9	1	11110
4	2	11111

Step 6: The resultant binary sequence is undergone on ASCII encoding technique

11010001 00111100 11100100 01110101 100110001
11011111 11010011 111010 000 (appended bits)

No.of ASCII characters= 8

Step 7: Calculate the compression ratio

Compression ratio= $8 \times 8 / 50 = 1.28$

IV. RESULTS AND DISCUSSIONS

DNA input data for experimentations is taken from GenBank [4]. Genbank is a popular public database where the human genomic data is available. GenBank is a part of the International Nucleotide Sequence Database Collaboration, which comprises the DNA DataBank of Japan (DDBJ), the European Molecular Biology Laboratory (EMBL), and GenBank is provided by at NCBI (National Center for Biotechnology Information). These three organizations exchange data on regular basis. So the data in GenBank increases exponentially. The performance of proposed

technique is verified on various real DNA sequences. The sequences used were three of human genes; HUMDYSTROP (human dystrophin gene), HUMHBB (human globin gene), HUMHDABCD (human sequence of contig); complete genome of mitochondria MPOMTCG and complete genome of vaccinia virus VACCG.

The Table 2 and graph in Fig.2 shows the comparison among the various existed DNA sequence compression algorithms like Biocompress2 [8], DNAcompress[9], Gencompress[10], CTW+LZ[11],DNA pack. The proposed technique gives effective compression ratios than existed algorithms. It can be noticed that data pertaining to HUMHDABCD shows good result using this technique. The average compression ratio is higher than the other compression algorithms.

Average compression ratio= 1.37 bits/base

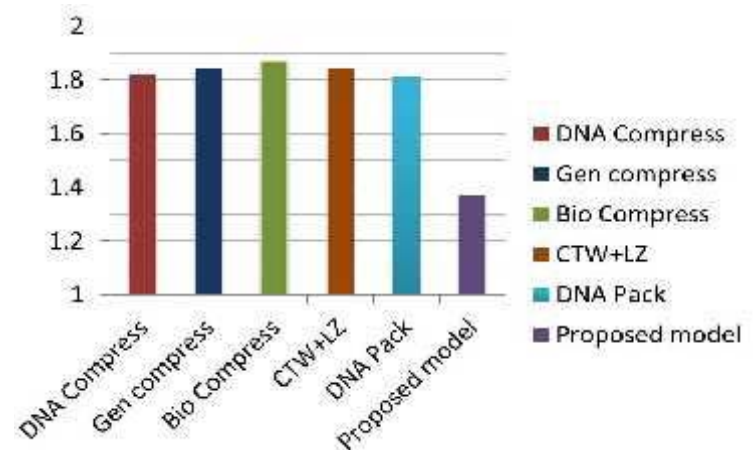


Fig2: Comparison with other methods

Table 2: Comparison with Other DNA Compression Algorithms (Bits/Base)

Sequence	Size	DNA Compress	Gen Compress	BioCompress 2	CTW+LZ	DNA Pack	Proposed model
HUMDYSTROP	38770	1.911	1.923	1.926	1.918	1.900	1.41
HUMHBB	73308	1.789	1.820	1.880	1.808	1.777	1.40
HUMHDABCD	58864	1.795	1.819	1.877	1.821	1.739	1.33
MPOMTCG	186609	1.892	1.901	1.937	1.900	1.893	1.36
VACCG	191737	1.758	1.761	1.761	1.761	1.758	1.37

References

- [1] Rao S and Bhat P Evaluation of lossless compression techniques In Communications and Signal Processing (ICCSP), 2015 International Conference on (pp. 1655-1659). IEEE.
- [2] Sayood K. Introduction to data compression. Newnes, 2012 Oct 4.
- [3] S. Grumbach and F. Tahi, Compression of DNA Sequences, Proc. IEEE Symp. Data Compression, Snowbird, UT, pp. 340-350, 1993.
- [4] Ms.Priyanka, Dr.Savitha Goel, A compression algorithm for DNA that uses ASCII value in IEEE International Advance Computing Conference (IACC),2014.

- [5] Jichao Ouyang, Ping Feng and Jichang Kang, Fast Compression of Huge DNA Sequence Data, 5th International Conference on Biomedical Engineering and Informatics (BMEI), 2012.
- [6] Sharma M. Compression using Huffman coding,(IJCSNS) International Journal of Computer Science and Network Security. 2010 May;10(5):133-41.
- [7] R.S.Aarthi, D. Muralidharan, P. Swaminathan, Double compression of test data using Huffman code, Journal of Theoretical and Applied Information Technology 15 May 2012.
- [8] Grumbach, S. and Tahi, F, A new challenge for compression algorithms: genetic sequences, Information Processing & Manegement, 30:875-886, 1994.
- [9] Chen, X., Li, M., Ma, B. and Tromp, J, DNACompress: fast and effective DNA sequence compression, Bioinformatics Application Note, 18(12):1696-1698, 2002.
- [10] X. Chen, S. Kwong and M. Li, A Compression Algorithm for DNA Sequences and its Applications in Genome Comparison, Proc.4thAnn.International Conf. Computational Molecular Biology, pp. 107, 2000.
- [11] Matsumoto T., Sadakane K., Imai H, Biological sequence compression algorithms. Genome Inform. Ser. Workshop GenomeInform.11:43-52,2000.